

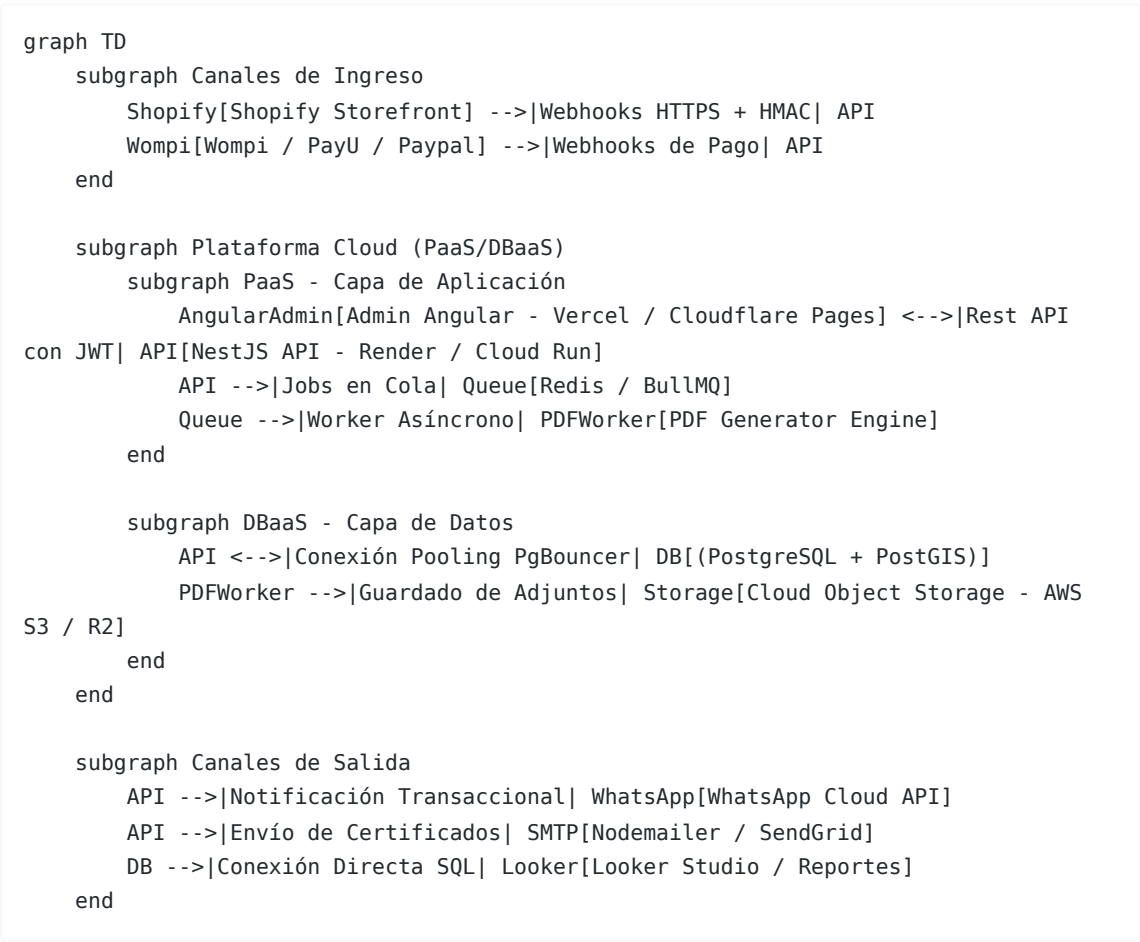
Plan Técnico de Sprints Semanales (Optimizado y Mitigado) - Centralización de Procesos

Cliente: Fundación Infantil Santiago Corazón

Rol: Staff Cloud Architect & Senior Technical Project Manager

- **Plazo Total:** 5 Meses (20 semanas de 1 semana por sprint).
- **Enfoque de Desarrollo:** Solo Developer (Full-stack + DevOps) → Requiere automatización máxima de infraestructura (PaaS/DBaaS) para eliminar sobrecargas operativas de administración de servidores.
- **Stack Definido:** Angular (v18+) | Node.js (NestJS / TypeScript) | PostgreSQL con PostGIS | PaaS (Render/Railway/GCP Cloud Run) | DBaaS (AWS RDS/GCP Cloud SQL/Neon).

1. Mapeo de Arquitectura Cloud de Alta Disponibilidad



2. Plan de Sprints Semana a Semana (Sprints 1 a 20)

 **SPRINT 1 (Semana 1):** Cimientos Cloud, DBaaS y Estructura Angular Base

[!IMPORTANT] Mitigación DevOps: Se descarta la configuración manual de un VPS (SSH, Nginx, cortafuegos manuales). En su lugar, se automatiza la infraestructura mediante un PostgreSQL gestionado (DBaaS) y despliegue continuo en PaaS. **Mitigación Migración:** Se adelanta el análisis y la ingeniería inversa del archivo de Access en el Sprint 1 para evitar cuellos de botella en el Sprint 6.

- **Tarea 1.1 [DevOps/BD] - Aprovisionamiento del DBaaS PostgreSQL**
 - *Descripción:* Crear e instanciar la base de datos en un servicio gestionado (AWS RDS, GCP Cloud SQL o Neon). Habilitar la extensión **PostGIS** (`CREATE EXTENSION postgis;`) para la geolocalización avanzada del Sprint 19. Configurar límites de pool de conexiones mediante PgBouncer.
 - *DoD (Definición de Done):* Conexión exitosa al DBaaS utilizando SSL forzado y autenticación segura.
- **Tarea 1.2 [Backend] - Configuración de Migraciones ORM (Prisma)**
 - *Descripción:* Inicializar Prisma en el backend de NestJS. Diseñar e implementar el esquema SQL para las tablas base: `roles` , `permissions` y `role_permissions` .
 - *DoD:* Migración inicial ejecutada y verificada en la base de datos cloud mediante `npx prisma migrate status` .
- **Tarea 1.3 [Data] - Ingeniería Inversa y Perfilado de Access**
 - *Descripción:* Extraer el esquema relacional y un volcado de prueba de la base de datos Microsoft Access. Mapear inconsistencias de datos, columnas nulas y definir la matriz de homologación hacia el nuevo esquema.
 - *DoD:* Documento de mapeo de datos Access-Postgres aprobado y tipos de datos de origen definidos.
- **Tarea 1.4 [Frontend] - Inicialización de Angular 18**
 - *Descripción:* Crear el frontend Angular con enrutamiento y Tailwind CSS. Configurar ambientes (`environment.ts`) dinámicos.
 - *DoD:* Aplicación Angular desplegada en Vercel o Cloudflare Pages con CI/CD automático desde la rama `main` de GitHub.

SPRINT 2 (Semana 2): Boilerplate API, CI/CD Automático y Capa de Comunicaciones Angular

- **Tarea 2.1 [DevOps] - Pipeline CI/CD del Backend**
 - *Descripción:* Configurar despliegue continuo (CD) en la plataforma PaaS (ej. Render/Railway) enlazado al repositorio GitHub. Al hacer push a `develop` , la plataforma compila el Dockerfile de NestJS de forma automática.
 - *DoD:* Push a la rama de git desencadena un deploy automático exitoso visible por URL pública HTTPS.
- **Tarea 2.2 [Backend] - Estructura de NestJS y Manejador Global de Excepciones**
 - *Descripción:* Configurar la arquitectura interna de NestJS (Modules, Controllers, Services, Repositories). Crear un filtro global de excepciones para registrar errores inesperados del sistema en la tabla `audit_logs` .
 - *DoD:* Peticiones fallidas devuelven un error controlado estándar, previniendo la fuga de información interna de base de datos.
- **Tarea 2.3 [Frontend] - Esqueleto Administrativo y Captura de Errores**
 - *Descripción:* Diseñar la estructura de módulos Angular y el interceptor HTTP global. Si la API backend retorna errores 400/500, el cliente los intercepta y los muestra mediante notificaciones Toast.

- *DoD*: Un request fallido simulado dispara la alerta toast en pantalla.
 - **Tarea 2.4 [Data] - Limpieza Previa del Dataset Histórico**
 - *Descripción*: Crear scripts de limpieza en Node/Python para normalizar los números telefónicos y estructurar las direcciones inválidas de Access basándose en la geografía local.
 - *DoD*: Dataset histórico exportado a CSV limpio listo para el cargador.
-

SPRINT 3 (Semana 3): Base de Datos de Usuarios y Autenticación JWT

- **Tarea 3.1 [Backend] - Esquema de Usuarios e Inserción de Roles**
 - *Descripción*: Modelar en Prisma la tabla `users` (`id`, `email`, `password_hash`, `role_id`, `is_active`). Ejecutar script de seed para roles (`Administrador` , `Director` , `Operador Tienda` , `Facturador`).
 - *DoD*: Llave foránea verificada y tabla poblada.
 - **Tarea 3.2 [Backend] - Endpoint de Registro y Encriptación Bcrypt**
 - *Descripción*: Implementar `POST /api/auth/register` . Cifrar las contraseñas usando `bcrypt` con un factor de trabajo salt de 10.
 - *DoD*: Las contraseñas en la base de datos son de solo lectura mediante hash irreversible.
 - **Tarea 3.3 [Backend] - Autenticación y Generación de Tokens JWT**
 - *Descripción*: Implementar `POST /api/auth/login` . Al ingresar credenciales correctas, firmar un token JWT (expiración: 8 horas) que incluya el ID del usuario y su rol.
 - *DoD*: Endpoint retorna token JWT y datos públicos de perfil en formato JSON estándar.
 - **Tarea 3.4 [Frontend] - Servicio Angular de Sesión y Reactive Login**
 - *Descripción*: Desarrollar la pantalla de Login reactiva y el servicio `AuthService` para guardar el JWT en `localStorage` .
 - *DoD*: Formulario bloquea el botón de envío si los campos son inválidos.
-

SPRINT 4 (Semana 4): Control de Accesos por Roles y Layout Base

- **Tarea 4.1 [Frontend] - Guardias de Rutas y Menú Lateral Adaptativo**
 - *Descripción*: Crear `AuthGuard` y `RoleGuard` en Angular para restringir accesos según roles decodificados del JWT. Construir el layout administrativo modular (sidebar, navbar, breadcrumbs).
 - *DoD*: Intentar acceder a `/admin` sin sesión redirige a `/login` .
 - **Tarea 4.2 [Frontend] - Interceptor para Adjuntar Token JWT**
 - *Descripción*: Programar un interceptor HTTP en Angular que agregue de forma automática la cabecera `Authorization: Bearer <token>` en todas las llamadas API salientes.
 - *DoD*: El backend recibe e identifica el token JWT en las cabeceras HTTP de Angular.
 - **Tarea 4.3 [Backend] - Gestión de Cuentas Administrativas**
 - *Descripción*: Endpoint `GET /api/users` (con filtros de rol) y `PUT /api/users/:id` protegidos por rol de Administrador.
 - *DoD*: Peticiones desde un token no administrador reciben código 403 Forbidden.
-

SPRINT 5 (Semana 5): Módulo CRM (Clientes y Donantes)

- **Tarea 5.1 [Backend] - Estructura SQL del CRM**
 - *Descripción*: Modelar la tabla `clients_donors` (`id`, `name`, `doc_type`, `doc_number`, `phone`, `email`, `address`, `city`, `commune`, `neighborhood`, `status`, `historical_id`, `created_at`). Crear índices en `doc_number` e `email` .

- *DoD*: Tabla creada e indexada en PostgreSQL.
 - **Tarea 5.2 [Backend] - Endpoints CRUD del CRM**
 - *Descripción*: Desarrollar endpoints `GET /api/crm/clientes` (con paginación y búsqueda por identificación), `POST /api/crm/clientes`, y `PUT /api/crm/clientes/:id`.
 - *DoD*: Búsqueda responde en menos de 200ms en base de datos.
 - **Tarea 5.3 [Frontend] - Interfaz Angular de Gestión de Clientes**
 - *Descripción*: Diseñar la grilla de CRM con paginación integrada al backend, filtros en tiempo real y formulario reactivo para creación de nuevos contactos.
 - *DoD*: Clic en guardar crea el cliente y refresca la tabla.
-

SPRINT 6 (Semana 6): Ejecución de la Migración Access (ETL)

- **Tarea 6.1 [Data] - Script ETL de Carga Masiva**
 - *Descripción*: Desarrollar el cargador final en Node/Python que lea el dataset pre-limpiado y realice inserciones masivas (Bulk Insert) a la tabla `clientes_donors` en PostgreSQL, utilizando transacciones de base de datos.
 - *DoD*: Script inserta miles de registros en menos de 5 minutos de forma íntegra.
 - **Tarea 6.2 [Data/QA] - Validación de Integridad de Datos**
 - *Descripción*: Desarrollar consultas de validación de recuentos e integridad para comprobar que la cantidad de registros en Access corresponde exactamente a los importados en PostgreSQL.
 - *DoD*: Reporte de validación de migración firmado sin discrepancias de registros.
 - **Tarea 6.3 [Frontend] - Ficha de Cliente y Perfil Histórico**
 - *Descripción*: Diseñar la pantalla de detalles del cliente para visualizar su información de contacto detallada.
 - *DoD*: Ficha del cliente carga e integra el historial del usuario migrado.
-

SPRINT 7 (Semana 7): Modelado de Pedidos y Transacciones ACID

- **Tarea 7.1 [Backend] - Migración SQL de Pedidos**
 - *Descripción*: Crear migraciones para las tablas `orders` y `order_items`.
 - `orders` : id, client_id, order_date, status, total_amount, payment_status, source, created_by.
 - `order_items` : id, order_id, product_id, quantity, unit_price, subtotal.
 - *DoD*: Integridad referencial configurada con borrado restringido (`ON DELETE RESTRICT`).
 - **Tarea 7.2 [Backend] - Lógica Transaccional ACID para Pedidos**
 - *Descripción*: Programar el servicio de creación de pedidos en el backend de forma que la orden principal y sus líneas de detalle (`order_items`) se inserten de forma atómica en una transacción.
 - *DoD*: Si ocurre un fallo al insertar un producto inválido, la base de datos realiza un rollback completo y no almacena ningún registro huérfano.
 - **Tarea 7.3 [Frontend] - Módulo Angular de Pedidos**
 - *Descripción*: Configurar el módulo Lazy-loaded `orders` y definir rutas para el listado general e ingreso manual.
 - *DoD*: Módulo registrado y accesible desde la barra lateral.
-

SPRINT 8 (Semana 8): Interfaz Angular para Pedidos Manuales

- **Tarea 8.1 [Frontend] - Formulario Dinámico de Pedido con Autocompletado**
 - *Descripción:* Desarrollar el formulario en Angular con un buscador autocompletado del CRM. Al seleccionar el cliente, carga sus datos de dirección de forma automática.
 - *DoD:* Escribir en el buscador filtra clientes existentes y permite seleccionarlos con un clic.
- **Tarea 8.2 [Frontend] - Selector Dinámico de Productos y Cálculo de Totales**
 - *Descripción:* Implementar una grilla dinámica en el formulario para agregar productos de un selector, cambiar cantidades numéricas y calcular subtotales y total general de forma reactiva.
 - *Criterio de Aceptación:* Modificar la cantidad de un ítem recalcula el total a pagar instantáneamente en pantalla.
- **Tarea 8.3 [Backend] - Endpoint de Consulta e Historial de Pedidos**
 - *Descripción:* Desarrollar `GET /api/orders` con filtros por fecha de creación, origen del pedido (Shopify, Manual, TiendaFísica) y estado de entrega.
 - *Criterio de Aceptación:* API retorna el JSON paginado de pedidos de manera correcta.
- **Tarea 8.4 [Frontend] - Listado y Gestión de Estados del Pedido**
 - *Descripción:* Diseñar el listado general de pedidos en Angular, incluyendo una columna interactiva para actualizar el estado del pedido (Recibido, En preparación, Despachado, Entregado).
 - *Criterio de Aceptación:* Cambiar el estado de un pedido desde la tabla de Angular actualiza el registro en la base de datos y refresca la vista.

17 **SPRINT 9 (Semana 9): Shopify Webhook & Pasarela Wompi (Fase 1 - Core de Ingresos)**

[[IMPORTANT] Mitigación Cuello de Botella: Para evitar sobrecargar el desarrollo en una única entrega, el Sprint 9 prioriza el Core de ingresos (Shopify + Wompi). Las demás pasarelas se desplazan a sprints posteriores.

Mitigación Tolerancia a Fallos: Se implementan mecanismos de **Idempotencia** (evitar duplicar pedidos si Shopify reintenta el webhook) y validación criptográfica de firmas.

- **Tarea 9.1 [Backend] - Endpoint de Shopify Webhooks y Firma HMAC**
 - *Descripción:* Desarrollar `POST /api/webhooks/shopify/orders`. Validar firma criptográfica usando header `X-Shopify-Hmac-SHA256` y llave secreta de Shopify.
 - *DoD:* El backend bloquea peticiones externas que no cuenten con firma de Shopify válida (retorna 401).
- **Tarea 9.2 [Backend] - Mecanismo de Idempotencia para Compras**
 - *Descripción:* Al recibir la orden, verificar si la identificación única de Shopify ya existe en la base de datos. Si existe, descartar procesamiento para evitar duplicados en base de datos.
 - *DoD:* Enviar el mismo payload dos veces consecutivas registra una sola orden en base de datos.
- **Tarea 9.3 [Backend] - Webhook de Pasarela Wompi (Donaciones Core)**
 - *Descripción:* Implementar `POST /api/webhooks/wompi`. Escuchar transacciones aprobadas, crear/asociar el donante en `clients_donors` e insertar el registro en `donations` de forma transaccional.
 - *DoD:* Recibir pago aprobado de Wompi actualiza de forma segura la base de datos local.

17 **SPRINT 10 (Semana 10): Procesamiento de Pedidos Shopify y Logs de Integración**

- **Tarea 10.1 [Backend] - Mapeo de Payload Completo Shopify**
 - *Descripción:* Lógica de procesamiento de pedidos Shopify: mapear variantes de producto, SKU, dirección de entrega y asociar al cliente por email.

- *DoD*: Pedidos de Shopify se listan en el panel administrativo de Angular con origen `Shopify` de forma correcta.
- **Tarea 10.2 [Frontend] - Dashboard de Logs y Errores de Sincronización**
 - *Descripción*: Diseñar pantalla en Angular para ver logs de Webhooks fallidos de Shopify/Wompi, detallando el payload y el error del servidor, con opción de reintento manual.
 - *DoD*: Los errores de webhooks son visibles y depurables desde la interfaz administrativa de Angular.

17 **SPRINT 11 (Semana 11): Pasarelas de Pago Secundarias (Fase 2 - PayU, PayPal, Frecuenti)**

- **Tarea 11.1 [Backend] - Integración de Webhooks de PayU**
 - *Descripción*: Desarrollar `POST /api/webhooks/payu`. Validar la firma criptográfica combinando el API Key, merchantId, referenceCode y value de la transacción.
 - *DoD*: Transacciones de PayU aprobadas ingresan exitosamente a la base de datos de la fundación.
- **Tarea 11.2 [Backend] - Integración de API de PayPal**
 - *Descripción*: Implementar la captura de pagos de PayPal mediante la verificación asíncrona de Instant Payment Notification (IPN).
 - *DoD*: Pagos aprobados de PayPal registrados en la tabla `donations`.
- **Tarea 11.3 [Frontend] - Listado Unificado de Donaciones por Pasarela**
 - *Descripción*: Panel en Angular para ver donaciones en línea, con filtros por pasarela (Wompi, PayU, PayPal) y estados.
 - *DoD*: La tabla muestra y filtra las transacciones unificadas en tiempo real de forma correcta.

17 **SPRINT 12 (Semana 12): Motor de Certificados PDF (Asíncrono y Desacoplado)**

[!IMPORTANT] Mitigación Arquitectura: Generar PDFs en la misma petición HTTP de la API genera timeout y sobrecarga de CPU. Se implementa un **diseño asíncrono basado en cola de tareas (BullMQ + Redis)**. Los archivos se guardan en un **Cloud Object Storage (AWS S3 / Cloudflare R2)** en vez del almacenamiento local.

- **Tarea 12.1 [Backend/DevOps] - Setup de Cola de Tareas (BullMQ + Redis)**
 - *Descripción*: Instalar y configurar un contenedor Redis ligero y configurar la librería `BullMQ` en NestJS para procesar tareas en segundo plano.
 - *DoD*: Conexión y encolamiento de tareas de prueba verificado en logs del servidor.
- **Tarea 12.2 [Backend] - Worker Asíncrono de Generación de PDF**
 - *Descripción*: Desarrollar el worker encargado de procesar la cola. Lee la plantilla del certificado, genera el PDF y lo sube de forma automática a AWS S3 o Cloudflare R2.
 - *DoD*: Al crearse una donación, el worker genera el PDF en segundo plano y almacena la URL de S3 en la tabla `certificates` de Postgres.
- **Tarea 12.3 [Frontend] - Visualizador de Certificados y Descargas**
 - *Descripción*: Diseñar el listado de certificados en Angular conectando el botón de descarga directamente a la URL segura y temporal (Pre-signed URL) de S3.
 - *DoD*: Clic en descargar abre o descarga el PDF guardado en el Cloud Storage.

17 **SPRINT 13 (Semana 13): Automatización de Correos SMTP para Donantes**

- **Tarea 13.1 [Backend] - Integración de Servicio de Correo SMTP**

- *Descripción:* Configurar la librería Nodemailer en backend apuntando a las credenciales de correo SMTP de la fundación.
 - *Criterio de Aceptación:* Correo de prueba se envía y recibe de forma correcta con adjuntos.
 - **Tarea 13.2 [Backend] - Disparador de Envío Automático al Donar**
 - *Descripción:* Desarrollar el evento en el backend: tan pronto el webhook de una pasarela confirma un pago exitoso y se genera el PDF, el sistema dispara el envío del correo de agradecimiento con el PDF adjunto de manera automática.
 - *Criterio de Aceptación:* Una simulación de webhook de Wompi exitosa culmina con la recepción automática del correo en la bandeja de entrada del donante simulado.
 - **Tarea 13.3 [Frontend] - Historial de Envíos y Reenvío Manual**
 - *Descripción:* Diseñar en la interfaz de Angular una columna que muestre si el certificado fue enviado con éxito por correo y un botón para forzar un reenvío manual inmediato si es necesario.
 - *Criterio de Aceptación:* Clic en "Reenviar" ejecuta una petición a la API y notifica visualmente del éxito del reenvío del correo.
-

SPRINT 14 (Semana 14): Catálogo de Inventario, Insumos y Recetas

- **Tarea 14.1 [Backend] - Esquema SQL de Inventarios**
 - *Descripción:* Diseñar las tablas en PostgreSQL para el control de stock: `products`, `inputs` (materias primas), `companions` (bolsas, moños, envolturas), y `product_inputs` (tabla que define cuántos gramos/unidades de cada insumo requiere un producto terminado).
 - *Criterio de Aceptación:* Tablas creadas e integradas relacionamente con integridad referencial activa.
 - **Tarea 14.2 [Backend] - CRUD del Catálogo de Inventario**
 - *Descripción:* Escribir endpoints para administrar el inventario físico: `GET /api/inventory/items`, `POST /api/inventory/items` y `PUT /api/inventory/items/:id`.
 - *Criterio de Aceptación:* API responde con el listado de productos, insumos y acompañantes.
 - **Tarea 14.3 [Frontend] - Módulo Angular de Inventarios y Tablas**
 - *Descripción:* Diseñar en Angular la tabla visual de productos, materias primas y acompañantes con sus SKU, existencias y estados.
 - *Criterio de Aceptación:* Interfaz muestra las listas de elementos reales en la base de datos de manera correcta.
-

SPRINT 15 (Semana 15): Ajustes Manuales y Alertas de Stock Mínimo

- **Tarea 15.1 [Backend] - Tabla y Lógica de Movimientos de Inventario**
 - *Descripción:* Crear la tabla `inventory_movements` (`id`, `item_type`, `item_id`, `movement_type` [Entrada, Salida, Ajuste], `quantity`, `user_id`, `description`, `timestamp`).
 - *Criterio de Aceptación:* Cada ajuste manual de stock realizado por un usuario registra un log de auditoría detallado en la tabla.
- **Tarea 15.2 [Frontend] - Modal para Registrar Ajuste Manual de Inventario**
 - *Descripción:* Desarrollar un formulario modal interactivo en Angular que permita al operador seleccionar un insumo o producto, ingresar una cantidad y registrar un tipo de ajuste (ej: pérdida por rotura).
 - *Criterio de Aceptación:* Enviar el formulario descuenta o suma el stock en base de datos y recarga la tabla en la interfaz.

- **Tarea 15.3 [Frontend] - Sistema de Alertas Visuales por Stock Mínimo**
 - *Descripción:* Programar la tabla de Angular para que compare la columna `stock` frente a `min_stock`. Si el valor es menor, colorea la fila en rojo e incluye un icono de alerta visual.
 - *Criterio de Aceptación:* Productos o insumos con stock crítico resaltan de forma llamativa al cargar el panel de inventario.

SPRINT 16 (Semana 16): Descuento Automático Transaccional de Stock (Concurrente y Seguro)

[!IMPORTANT] Mitigación Concurrency: Compras simultáneas en Shopify de un producto con stock limitado pueden causar stock negativo (Condiciones de carrera). Se implementa bloqueo pesimista mediante consultas nativas `SELECT ... FOR UPDATE` a nivel de base de datos PostgreSQL en la transacción.

- **Tarea 16.1 [Backend] - Servicio Transaccional de Descuento de Existencias**
 - *Descripción:* Desarrollar una transacción SQL que busque y bloquee la fila del producto (`SELECT * FROM products WHERE id = ? FOR UPDATE`). Si hay disponibilidad, descuenta el stock de productos, insumos y acompañantes.
 - *DoD:* Si no hay stock suficiente de insumos en la receta, la transacción falla, hace rollback completo y cancela el cambio de estado del pedido.
- **Tarea 16.2 [Backend/QA] - Pruebas de Carga de Concurrency**
 - *Descripción:* Escribir pruebas unitarias / de integración simulando 10 peticiones simultáneas de descuento de stock de un producto con existencias en valor 1.
 - *DoD:* Solo 1 petición debe tener éxito; las 9 restantes deben rebotar con error controlado de stock insuficiente.
- **Tarea 16.3 [Frontend] - Indicador Visual de Stock y Advertencia**
 - *Descripción:* Alertas visuales y bloqueos de botones de envío en Angular si el sistema detecta que el inventario local no puede cumplir con el pedido manual.
 - *DoD:* El operador de tienda física visualiza advertencia clara y no puede forzar despachos sin inventario disponible.

SPRINT 17 (Semana 17): Notificaciones por WhatsApp Business API

Lista de Tareas del Sprint 17:

- **Tarea 17.1 [Backend] - Integración del SDK/Client de WhatsApp API**
 - *Descripción:* Programar en backend la integración para realizar llamadas HTTP POST seguras a la API de WhatsApp Cloud (o proveedor Twilio) con tokens de portador permanentes.
 - *Criterio de Aceptación:* Envío exitoso de mensaje básico de prueba a número del desarrollador.
- **Tarea 17.2 [Backend] - Disparador de Alertas por Cambio de Estado de Pedidos**
 - *Descripción:* Configurar la llamada automática al servicio de WhatsApp cuando el estado del pedido pase a "Despachado", enviando un mensaje con los datos de transporte al teléfono del cliente.
 - *Criterio de Aceptación:* Cambiar el estado de un pedido en el panel de Angular envía la plantilla de WhatsApp correspondiente de forma inmediata.
- **Tarea 17.3 [Backend] - WhatsApp para Donantes con Enlace de Certificado**
 - *Descripción:* Programar el envío automático del mensaje de WhatsApp de agradecimiento al donante cuando el webhook aprueba su pago, incluyendo en el mensaje un enlace seguro de descarga directa del certificado PDF.

- *Criterio de Aceptación:* Una donación exitosa envía un mensaje que contiene el enlace al PDF del certificado de donación.

SPRINT 18 (Semana 18): Dashboard Privado y Métricas de Gestión

Lista de Tareas del Sprint 18:

- **Tarea 18.1 [Backend] - Queries SQL Agregadas para Ventas y Donaciones**
 - *Descripción:* Desarrollar las consultas SQL que calculen las ventas agrupadas por rangos temporales (día, mes, trimestre, semestre, año) e ingresos de donaciones segmentados por campaña activa y pasarela de pago.
 - *DoD:* Endpoint retorna datos consolidados históricos en menos de 200 ms.
- **Tarea 18.2 [Backend] - Consultas de Gestión Operativa (Pedidos e Inventario)**
 - *Descripción:* Escribir endpoints para obtener conteos de pedidos por estado, stock crítico actual de insumos, total de certificados generados frente a pendientes de envío, y auditorías de actividad de usuarios.
 - *DoD:* Datos se entregan agrupados en un solo JSON estructurado para el dashboard.
- **Tarea 18.3 [Backend] - Servicio de Filtros Multidimensionales**
 - *Descripción:* Programar filtros avanzados que admitan combinaciones de búsqueda: por canal de origen, rango de fechas, productos específicos, nombre de campaña, ubicación geográfica y tipo de cliente.
 - *DoD:* Filtrado compuesto en base de datos retorna resultados exactos sin degradar el rendimiento.
- **Tarea 18.4 [Backend] - Query de Productos con Mayor Rotación**
 - *Descripción:* Escribir consulta SQL que ordene los productos terminados e insumos por mayor cantidad vendida/consumida en un periodo dado.
 - *DoD:* Generación del top 10 de productos más vendidos consumible por la API.
- **Tarea 18.5 [Frontend] - Componentes de Gráficos de Gestión**
 - *Descripción:* Implementar gráficos interactivos en Angular (líneas, barras horizontales, diagramas de anillo) usando la librería `ngx-charts` conectada a los endpoints analíticos.
 - *DoD:* Gráficas pintan datos dinámicos y reflejan cambios al aplicar filtros.
- **Tarea 18.6 [Frontend] - Vista de Control y KPIs de Dirección**
 - *Descripción:* Diseñar la pantalla del Dashboard Administrativo organizando tarjetas de indicadores y selectores de rangos de fechas.
 - *DoD:* Dashboard interactivo totalmente funcional y responsivo.

SPRINT 19 (Semana 19): Dashboard Público y Visualización con Base de Datos Espacial (PostGIS)

[!IMPORTANT] Mitigación Arquitectura: Agrupar y buscar donantes por zonas geográficas de forma tradicional (por texto o rangos) en base de datos es ineficiente y propenso a errores. Se utiliza **PostGIS** para calcular áreas espaciales reales (polígonos de comunas/barrios) optimizado con **Índices Espaciales GIST**.

- **Tarea 19.1 [Backend] - Conversión de Direcciones a Coordenadas (Geocoding)**
 - *Descripción:* Implementar en la API backend un script que consuma la API de geocodificación de OpenStreetMap (Nominatim) o Google Maps para transformar las direcciones de los donantes migrados y nuevos en coordenadas `GEOMETRY(Point, 4326)`.

- *DoD:* Las direcciones de los donantes se almacenan con coordenadas geográficas correctas en la base de datos PostgreSQL.
- **Tarea 19.2 [Backend] - Consultas Espaciales por Polígonos de Comunas**
 - *Descripción:* Crear endpoints de consulta que usen funciones PostGIS como `ST_Contains` o `ST_DWithin` para cruzar la ubicación de los donantes frente a los polígonos geográficos de las comunas o barrios de la ciudad.
 - *DoD:* Retorna el conteo y densidad de donantes por comuna de forma anonimizada en formato GeoJSON.
- **Tarea 19.3 [Frontend] - Integración de Mapa Interactivo (Leaflet)**
 - *Descripción:* Configurar la librería Leaflet en Angular. Consumir el GeoJSON del backend y pintar el mapa de calor de donaciones acumuladas agrupadas en clusters.
 - *DoD:* El mapa renderiza los puntos geográficos de donaciones acumuladas de forma correcta y visualmente interactiva.
- **Tarea 19.4 [Frontend] - Landing Page Pública Embebible**
 - *Descripción:* Diseñar el Layout responsivo de la landing de impacto, integrando el mapa Leaflet, tarjetas de indicadores principales e históricos.
 - *DoD:* Vista funciona embebida en un Iframe dentro del sitio web principal de la fundación.

SPRINT 20 (Semana 20): Exportación Contable, QA y Despliegue Final

Lista de Tareas del Sprint 20:

- **Tarea 20.1 [Backend] - Generador de Plantilla CSV para World Office**
 - *Descripción:* Programar la lógica en backend para mapear los pedidos y donaciones de un periodo de fechas a un archivo CSV estructurado según el manual técnico contable de World Office.
 - *Criterio de Aceptación:* Exportación exitosa descargando el archivo CSV con las cabeceras e importación correcta de prueba en World Office.
- **Tarea 20.2 [DevOps] - Despliegue de Producción y Configuración SSL**
 - *Descripción:* Realizar el build de producción del panel en Angular y desplegar la aplicación y la API Backend en el servidor VPS definitivo. Configurar Let's Encrypt para HTTPS.
 - *Criterio de Aceptación:* El sistema completo opera bajo el dominio final seguro `https://admin.santiagocorazon.org` con SSL válido.
- **Tarea 20.3 [DevOps] - Tareas Programadas de Backup (Cron Jobs)**
 - *Descripción:* Crear e implementar un Cron Job a nivel de sistema operativo para realizar un backup diario automatizado (volcado sql) de la base de datos PostgreSQL y cargarlo en un almacenamiento seguro en la nube.
 - *Criterio de Aceptación:* Verificación de que el backup diario se crea y guarda en la nube correctamente.
- **Tarea 20.4 [QA / Gestión] - Pruebas de Sistema, Capacitación y Firma de Acta**
 - *Descripción:* Realizar pruebas funcionales cruzadas de todo el flujo del software. Capacitar al personal administrativo de la fundación y realizar la firma del acta de entrega.
 - *Criterio de Aceptación:* Todos los flujos funcionan sin fallos, el personal sabe registrar inventarios y pedidos, y el proyecto se marca como completado y entregado.